

Article

A Lightweight Malware Detection Model Based on Knowledge Distillation

Chunyu Miao ¹, Liang Kou ^{2,*}, Jilin Zhang ² and Guozhong Dong ^{3,*}¹ Research Center of Network Application Security, Zhejiang Normal University, Jinhua 321017, China² College of Cyberspace, Hangzhou Dianzi University, Hangzhou 310005, China³ Pengcheng Laboratory, Department of New Networks, Shenzhen 518066, China

* Correspondence: kouliang@hdu.edu.cn (L.K.); donggz@pcl.ac.cn (G.D.)

† These authors contributed equally to this work.

Abstract: The extremely destructive nature of malware has become a major threat to Internet security. The research on malware detection techniques has been evolving. Deep learning-based malware detection methods have achieved good results by using large-scale, pre-trained models. However, these models are complex, have large parameters, and require a large amount of hardware resources and have a high inference time cost when applied. To address this challenge, this paper proposes DistillMal, a new method for lightweight malware detection based on knowledge distillation, which improves performance by using a student network to learn valuable cueing knowledge from a teacher network to achieve a lightweight model. We conducted extensive experiments on two new datasets and showed that the student network model's performance is very close to that of the original model and the outperforms it on some metrics. Our approach helps address the resource constraints and computational challenges faced by traditional deep learning large models. Our research highlights the potential of using knowledge distillation to develop lightweight malware detection models.

Keywords: malware detection; pre-training models; knowledge distillation; lightweight models

MSC: 68T07



Citation: Miao, C.; Kou, L.; Zhang, J.; Dong, G. A Lightweight Malware Detection Model Based on Knowledge Distillation. *Mathematics* **2024**, *12*, 4009. <https://doi.org/10.3390/math12244009>

Academic Editor: António Lopes

Received: 10 November 2024

Revised: 10 December 2024

Accepted: 13 December 2024

Published: 20 December 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

With the increasingly frequent use of the Internet, the proliferation of backdoor software, ransomware, and other malware poses a great threat to people when accessing Internet. Therefore, improving the technology used to detect malware is an urgent requirement. In recent years, malware detection techniques based on deep learning have made great progress [1–3]; these techniques can effectively learn and detect the features of malware [4]. API sequences can be considered long sequences of combinations of multiple APIs. Therefore, when choosing a deep learning method to process an API sequence, the long-term dependency modeling ability of the model needs to be considered. Based on this property of API sequences, researchers added the Transformer structure to the traditional malware detection methods based on RNN to better capture the dependencies between long sequences using the self-attention mechanism [5,6]. BERT [6] is a multi-layer Transformer module stacked model that is widely used for pre-trained language representation, which is trained with large amounts of text data and can capture rich contextual information. The researchers usually use Natural Language Processing (NLP) techniques to analyze the API features, while pre-trained large-scale models such as BERT [6], GPT [7], etc., can achieve good detection with a simple fine-tuning due to their ability to be pre-trained on large-scale databases. However, these methods suffer from an excessive number of model parameters and a long inference time, which limits their application in practical detection. To address the above problems, we propose a novel malware API sequence detection model using knowledge distillation techniques.

Malware detection methods based on transfer learning can solve this problem. They use a pre-trained CNN network as the initial network and then transfer to the target network [8–10]. Knowledge distillation is an effective transfer learning method designed to improve the performance of a student model by extracting knowledge from a large, complex model (the teacher model) and transferring it to a small, simplified model (the student model) [11]. This technique has been successfully applied in many fields, such as image recognition and object detection [12,13]. In malware detection, knowledge distillation can be used to train a smaller, more efficient model that can run on resource-limited devices, boosting its utility [14].

Our approach first trains a large deep neural network (BERT) on a large dataset to detect malware. We then use knowledge distillation to transfer the knowledge learned by the large model to a smaller model (TextCNN) that can be deployed on resource-constrained devices. Our approach has several advantages over traditional malware detection approaches for IoT security.

First, our approach uses contrast learning to capture the features of API sequences. Contrast learning is typically used to learn valid feature representations of images, text, or other types of data [15,16]. Richer feature representations can be learned than when using other methods, and the model's performance in downstream tasks can be improved [17].

Second, our approach reduces the time required and the computational cost of model through knowledge distillation. The knowledge distillation module maintains accuracy while reducing the number of parameters and the inference time by transferring knowledge from large models to lightweight models.

Overall, our approach provides an innovative solution for detecting malware in the presence of constrained computational resources. By using knowledge distillation techniques, we can improve the accuracy and efficiency of the model. The rest of this paper is organized as follows: Section 2 reviews the related research on malware detection; Section 3 introduces the proposed framework in detail; Section 4 shows the experimental results and evaluates the performance of the proposed method; and finally, Section 5 summarizes the whole paper and discuss the future research directions.

2. Related Work

Malware detection and analysis methods can be categorized into static analysis, which relies on static features, and dynamic analysis, which uses dynamic features. Static features are the program characteristics extracted through reverse analysis without executing the program. Li et al. [18] achieved software detection by taking the features of the software to be tested, such as the file header information and dynamic link libraries, as the input for the support vector machine (SVM). Ahmadi et al. [19] proposed a static learning-based malicious program classification method which directly extracts the features of the malicious program without the need for unpacking. Raman et al. [20] detected PE header information using an xp system as well as a Windows 7 system by means of random forest (RF) and a variant of decision trees (DTs). Kumar et al. [21] further combined their derived information with the original information using file headers with K-Nearest Neighbors (KNN), RF, and many other machine learning methods for malware detection. Since there is no need to run the corresponding program, the time and space complexities of the static analysis methods appear to be lower compared to those of dynamic analysis, which has a faster detection speed. However, with the enhancement of escape techniques, malware developers can bypass static detectors using a similar technique to code obfuscation, code deformation, and dynamic loading [22]. Singh et al. [23] set up a dynamic analysis environment using the Cuckoo sandbox to extract features at various call runtimes and classify them using the random forest approach. The development of deep learning enriched the dynamic feature-based malware detection methods [24–31]. Ki et al. [25] employed a DNA sequence comparison algorithm to extract common API calls associated with malicious behaviors across various malware categories. Using the extracted API sequences, they developed a signature-based malware detection mechanism, demonstrating

through experiments that these API sequences are effective in characterizing specific types of malicious activities. Huang et al. [26] proposed a novel malware detection approach that integrates malware visualization techniques with convolutional neural networks (CNNs). In this approach, malware samples are first dynamically analyzed within a Cuckoo sandbox environment. The dynamic analysis results are then transformed into visual representations, which are subsequently classified using a VGG16 network trained on the image data. Catak et al. [27] utilized N-gram and Term Frequency-Inverse Document Frequency (TF-IDF) methods for feature extraction and selection, applying a two-layer Long Short-Term Memory (LSTM) model to capture the temporal relationships between the API calls in a sequence. Xu et al. [30] introduced the MalBERT framework, which leverages a pre-trained model based on the Transformer architecture. Compared to traditional LSTM models and other machine learning approaches, the MalBERT framework achieved a superior performance in terms of malware detection, demonstrating the advantages of pre-training to improve feature extraction and classification. Large models achieve good results at the cost of high computational and time costs; therefore, a lightweight model based on knowledge distillation is proposed. The model improves the efficiency while maintaining the original accuracy [32].

3. Methodology

3.1. Framework

Based on the problems of directly applying the BERT model to API sequence detection, we proposed DistillMal, a lightweight malware API sequence detection framework based on knowledge distillation. Figure 1 illustrates the framework of DistillMal. First, we cleaned the original API sequence to remove duplicates and irrelevant information. Subsequently, the cleaned API sequences were fed into the embedding layer to transform the API sequences into vector forms that can be processed by the model. Based on the output of the embedding layer, we further fine-tuned the pre-trained teacher model. During the fine-tuning process, we introduced a contrastive learning task, which allowed the teacher model to learn the features of the malware samples more deeply, thus improving its performance. After the fine-tuning of the teacher model was completed, our student model updated the network through a knowledge distillation process to simulate the soft-label output distribution of the teacher model as well as to provide a comparison with real hard labels.

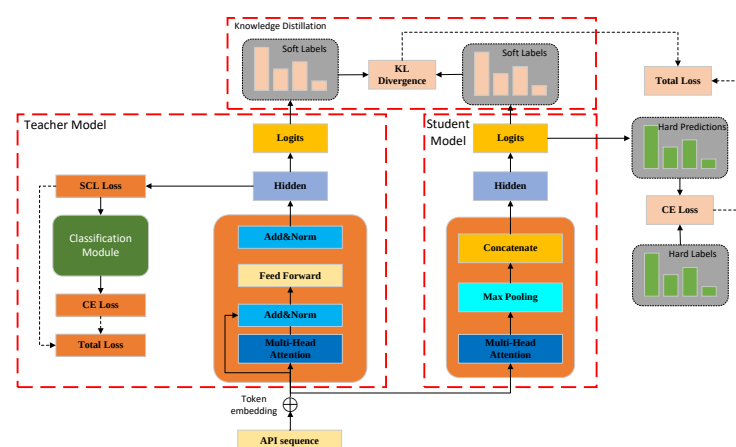


Figure 1. The overall structure of DistillMal.

3.2. Data Pre-Processing and Embedding

To avoid being analyzed, malicious code often inserts a large number of redundant operations. Also, looping statements in API call sequences can cause the code to repeat certain commands. This redundant information not only disrupts the analysis process, but also increases the time cost of training. So, when preprocessing API sequences, only

the non-repetitive parts of multiple consecutive calls to the same API are retained, and redundant subsequences are removed. After the de-duplication process, the API sequences need to be transformed into a format suitable for BERT input. In BERT's normalized input format, the length of the sequence first needs to be fixed, with shorter sequences padded and longer sequences cut. Then, [CLS] and [SEP] tags are added on both sides of the sequence to identify the start and end of the sequence.

After going through the preprocessing operations, the API sequence needs to be converted to vector form. The embedding layer consists of three parts: Token Embedding, Segment Embedding, and Position Embedding. The overall embedding process is shown in Figure 2. Token Embeddings convert each API in an API sequence into a vector representation. Each API corresponds to a unique token. Segment Embeddings help the model understand the relationships between different API sequences. By adding Segment Embeddings, the model can distinguish between APIs from different API sequences and parse their semantics correctly in context. Position Embeddings are used to capture information about the position of an API in an API sequence. They will assign a specific positional encoding to each API, thus telling BERT the exact position of the API in the sequence.

This embedding layer helps the model to understand the semantic and contextual information of the APIs so that it can better capture the relationships between APIs. By embedding the input API sequences, the model can efficiently process each API and encode them into meaningful vector representations. After the embedding layer, the input matrix $E = \{E_1, E_2, \dots, E_N\}$ into the coding layer can finally be obtained.

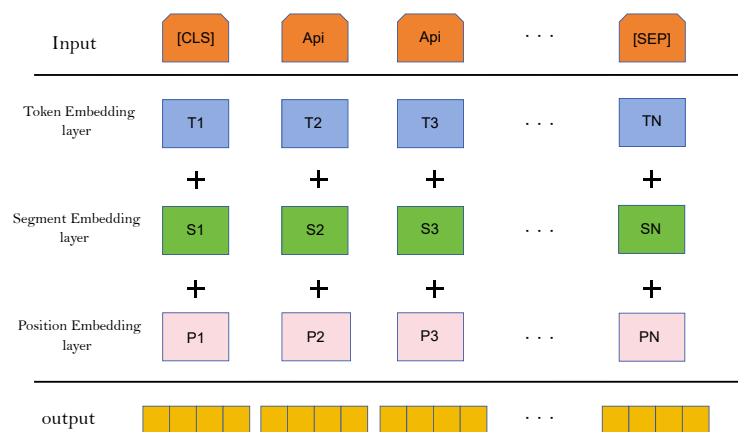


Figure 2. Model embedding flowchart.

3.3. The Pre-Trained Model Fine-Tuning

Teacher model Bert's encoder is a stacked transformer. Through the multi-head attention mechanism, the model realizes parallel attention to the input sequence and collects and integrates information from the different attention heads. This mechanism enables the model to process multiple related but independent tasks or concerns simultaneously. After obtaining the model encoder layer's output, the BERT model is fine-tuned through the addition of a supervised contrastive learner to improve BERT's performance in the API sequence detection task. Due to the specialized nature of the malware API sequence vocabulary, which is more low-frequency than the overall vocabulary, the representation of word vectors may not be spatially distributed in a desirable way, while contrastive learning can reduce the various anisotropies [33]. According to the results of experiments conducted by Beliz Gunel et al. [34], the addition of contrastive loss can improve the performance and robustness of the model. Therefore, the contrastive learning task was added to optimize the word vector representation of API sequences and to improve the classification of the model. The overall flow of the fine-tuning is shown in Figure 3:

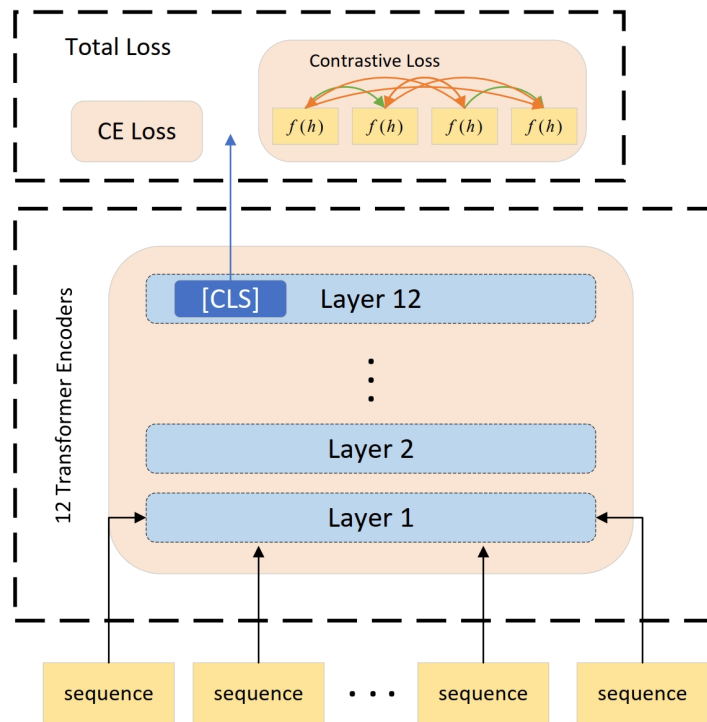


Figure 3. The pre-trained model's fine-tuning process.

Contrastive learning aims to learn efficient representations by pulling semantically close neighbors together and pushing away non-neighbors. P. Khosla et al. [35] found that label information can improve the accuracy of contrastive learning. Thus, supervised contrastive learning loss is used to fine-tune the processed API sequences. In supervised contrastive learning, the classes with the same label in a batch are usually considered positive classes, while the classes with different labels in a batch are considered negative classes.

For a malware classification task with C categories, in a training sample with batch size N containing samples $x_i, y_i, i \in 1, \dots, N$, the supervised contrastive loss with samples is calculated using Equation (1):

$$L_{SCL} = \sum_{i=1, i \neq j, i \neq k}^N -\frac{1}{N_{y_i} - 1} \sum_{j=1}^N \log \frac{e^{\left(\Phi(x_i) \cdot \frac{\Phi(x_j)}{\tau}\right)}}{\sum_{k=1}^N e^{\left(\Phi(x_i) \cdot \frac{\Phi(x_k)}{\tau}\right)}} \quad (1)$$

where N_{y_i} denotes the number of samples with the same label in the same batch, $\Phi(x_i)$ denotes the vector representation of the API sequence after the encoder. τ is a temperature parameter used to control the degree of separation between samples. By adjusting this parameter, the model's ability to learn from hard-to-distinguish samples can be influenced.

Then, the encoder feature extractor is fixed and then the classifier is trained. At this point, cross-entropy loss is used to train the classifier and calculate the difference between the probability distribution of the model output and the true label. The formula for the cross-entropy loss function is shown in Equation (2).

$$L_{CE} = - \sum_{i=1}^N y_i \log y'_i \quad (2)$$

where y_i is the real label and y'_i is the predicted value of the model; then, there is an overall loss, as follows.

$$L_{Total} = (1 - \lambda) L_{CE} + \lambda L_{SCL} \quad (3)$$

λ is a hyperparameter used to control the proportion of individual losses in the overall loss. The overall fine-tuning process is shown in Algorithm 1:

Algorithm 1 The pre-trained model fine-tuning process

```

1: Input: labeled API sequences
2: Output: the fine-tuned Bert model
3: Epochs  $\leftarrow$  define the number of epochs
4: Features, Labels  $\leftarrow$  process the API sequences and labels
5: Token sequences  $\leftarrow$  tokenizer(Features)
6: Model  $\leftarrow$  Bert model with classification module
7: while Epochs do
8:   for data in Token sequences do
9:     Hidden_Output, Classifier_Output = Model(data)
10:    CE_Loss = CrossEntropyLoss(Classifier_Output, Labels)
11:    SC_Loss = Supervised ContrastiveLoss(Hidden_Output)
12:    Total_Loss =  $(1 - \lambda) \cdot CE\_Loss + \lambda \cdot SC\_Loss$ 
13:    Backward and update parameter
14:   end for
15: end while

```

3.4. Knowledge Distillation

The fine-tuned model has the ability to categorize API sequences into different malware types, but still suffers from too large a parameter size and too long an inference time. To address this problem, we employed a model distillation technique to migrate the rich knowledge contained in the teacher's model to the more lightweight student model. We chose TextCNN as our student model, a convolutional neural network (CNN) architecture designed specifically for text categorization tasks. TextCNN has been widely used in the field of text processing due to its simple model structure and excellent classification effect. The model effectively extracts key features through a one-dimensional convolutional layer, and its feature learning ability is very good despite the simplicity of the model structure.

In the implementation of TextCNN, we used different convolution kernels of sizes 3, 4, and 5 to perform convolution operations on the input feature matrix. This process can be represented as follows:

$$C_i = \text{Relu}(W \cdot X_i + b) \quad (4)$$

where W represents the convolutional kernel, b is the bias term, and X_i represents the input vector. Convolutional kernels of different sizes can capture richer feature information, so the model can understand the input data more comprehensively. The Relu function was also introduced for nonlinear transformation, to allow the model to better adapt to a complex data distribution.

After the convolution operation, maximum pooling is introduced to reduce the parameters and computation of the network:

$$C_{max} = \text{MAX}(C_i) \quad (5)$$

The maximum pooling operation not only reduces the data dimensions but also retains the most representative feature information by selecting the maximum value in the feature map. Finally, the output is processed by the fully connected layer, as shown in Figure 4.

Both the teacher network and the student network received the API sequence features processed by the embedding layer as the input. In the knowledge distillation process, we first fixed the parameters of the teacher model to ensure that they remained constant during training. Subsequently, we input the API sequences into the BERT model to obtain the logits of the teacher model. These logits reflect the judgment of the teacher model regarding the information of the API sequences and serve as both soft and target labels for the student network. To measure the difference in the knowledge learned by the teacher model and the

student model, we used Kullback–Leibler (KL) scatter as a metric tool; the formulation is as follows:

$$L_{KL} = t^2 \sum_x P(x)^t \left[\log \frac{P(x)^t}{Q(x)^t} \right] \quad (6)$$

where $P(x)$ is the probability distribution of the soft labels of the teacher model, $Q(x)$ is the probability distribution of the soft labels of the student model, and t is a temperature parameter that controls the smoothing of the probability distribution. By minimizing the KL scatter loss, we were able to direct the student model $Q(x)$ to fit the probability distribution of the teacher model $P(x)$ so that the student network learned the output distribution of the teacher network.

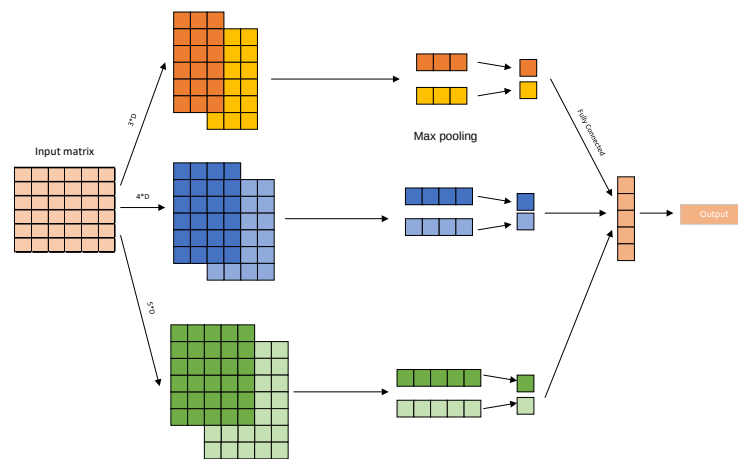


Figure 4. The framework of the student model TextCNN.

In addition to the learning process of knowledge distillation, the student network needs to learn through a comparison with real labels to ensure that it can acquire knowledge directly from the data. This process was realized through the cross-entropy loss function. The cross-entropy loss function was used to measure the difference between the classification results of the student model and the real labels. Combining the KL scatter loss for soft label differences and the cross-entropy loss for hard labels, we introduced a hyperparameter σ to balance the effects of these two parts of the loss. The total loss function of the student network is as follows:

$$L_{Total} = (1 - \sigma)l_{KL} + \sigma l_{CE} \quad (7)$$

4. Experiment

4.1. Experimental Setting

The hardware environment of the experimental equipment was as follows: CPU was 12 vCPU Intel(R) Xeon(R) Platinum 8255C CPU @ 2.50 GHz; GPU was RTX 3090. The VirusShare dataset and VirusSample dataset [36] were selected in this paper. To ensure the reliability of the experimental results, the dataset was randomly divided into training sets and testing sets in a ratio of 8:2. The following parameters were set in this paper: (1) the batch size was 32; (2) the fixed length of the API sequence was set to 512; (3) the dropout was set to 0.5; (4) the Adam optimizer was selected and the initial learning rate was 0.001; (5) the t was set to 5; (6) the λ was set to 0.9; (7) the σ was set to 0.7.

4.2. Dataset and Baseline

In the experiments using the DistillMal model, the VirusSample and VirusShare datasets were used as the experimental datasets. These datasets were recently created by researchers at Gebze Technical University and Kadir Has University, and were both constructed through the extraction of MD5 hash codes and API call sequences from malware

samples. The dataset contained multiple malware families. The dataset exhibited an issue where some malware categories had insufficient samples. Significant imbalances in the number of malware samples can affect the performance of multi-class classification models; therefore, in the data pre-processing stage, this study removed malware categories with fewer than 100 samples from the dataset and discarded the “undefined” malware category. This resulted in a dataset containing 13,849 and 9732 malware samples, as detailed in Table 1.

Table 1. Specific categorization of the datasets.

Malware Categories	Dataset	
	VirusShare	VirusSample
Trojan	8919	6153
Virus	2490	2367
Advertising Software	908	222
Back door	510	447
Download	218	N/A
Worm virus	524	300
Proxy	165	165
Ransomware	115	N/A
Total	13,849	9372

In order to validate the malware detection effect of the DistillMal model proposed in this paper, this paper used Accuracy, Macro-F1, Macro-Precision, and Macro-Recall as the evaluation metrics and set up the following baseline experiments.

MLP [37]: MLP(Multilayer Perceptron) is a feed-forward neural network model. It consists of multiple neural network layers, each of which consists of multiple neuron nodes. The MLP passes information between the input and output layers through multiple hidden layers and uses nonlinear activation functions for computation. The model is commonly used in classification, regression, and other machine learning tasks.

TextCNN [38]: TextCNN is a convolutional neural network text categorization model widely used in natural language processing. It uses a one-dimensional convolutional layer to extract local feature, and then selects the most salient features through a maximum pooling operation and classifies them through a fully connected layer. TextCNN captures the features of text fragments of different sizes and understands the context and semantics of the text. The number of parameters is small and the model is computationally fast.

Catak [27]: BiLSTM (Bidirectional Long Short-Term Memory Network) is a class of neural network architectures for sequence data processing. In contrast to classical LSTMs, BiLSTMs analyze the input sequence data by setting up a forward and a backward two-direction LSTM layer in parallel within its structure. This bi-directional processing allows BiLSTM to consider the contextual information from both ends of the sequence in a comprehensive manner, thus more accurately recognizing and grasping the dependencies in the sequence.

BERT-base [6]: BERT-base is a pre-trained language model based on Transformer. It learns semantic representations from a large-scale corpus through unsupervised learning and is capable of capturing the contextual relationships between words. The core idea of BERT-base is to use a bidirectional Transformer architecture that encodes the input sequence twice, from left to right and right to left, respectively, thus achieving a comprehensive understanding of the entire context. This bidirectional encoding strategy enables BERT-base to better capture semantic and syntactic information when dealing with natural language processing tasks.

DistillBert [39]: DistillBert is a knowledge distillation method based on BERT models, which improves the performance of pre-trained large BERT models by transferring their knowledge to small BERT models, reducing the number of parameters and the computational overhead of the models while maintaining a high level of performance.

4.3. Results and Analysis

In the model validity analysis experiments, five baseline models were compared and the overall performance was analyzed using four evaluation metrics. Tables 2 and 3 show the results of the multi-classification experiments for each model on the VirusSample and VirusShare datasets, respectively:

Table 2. Multi-classification results on VirusSample.

Metrics	Accuracy	M-Recall	M-Precision	M-F1
MLP	0.917	0.435	0.576	0.469
TextCNN	0.931	0.707	0.633	0.641
Catak	0.665	0.206	0.305	0.208
BERT-base	0.946	0.557	0.677	0.562
DistillBert	0.946	0.569	0.687	0.598
DistillMal (Our's)	0.942	0.501	0.734	0.601

Table 3. Multi-classification results on the VirusShare dataset.

Metrics	Accuracy	M-Recall	M-Precision	M-F1
MLP	0.835	0.479	0.576	0.511
TextCNN	0.884	0.627	0.633	0.678
Catak	0.663	0.283	0.374	0.303
BERT-base	0.892	0.652	0.878	0.701
DistillBert	0.890	0.651	0.876	0.698
DistillMal (Our's)	0.891	0.636	0.887	0.692

The experimental results show that the lightweight malware detection model based on knowledge distillation (DistillMal) proposed in this paper exhibited a superior classification performance in several evaluation metrics. On the VirusSample dataset, DistillMal performed better on the M-Precision and M-F1 metrics, even though it was slightly less accurate than BERT-base. On the VirusShare dataset, DistillMal outperformed DistillBert in accuracy and outperformed BERT-base in the M-Precision metric. A comparison with the experimental results of the TextCNN model shows that, through knowledge distillation, the TextCNN model successfully absorbed the task knowledge from the teacher model. This further confirms the effectiveness of knowledge distillation in model performance optimization.

Further analysis shows that the increase in model complexity helps to improve malware detection. Compared to the simple MLP model, more complex models such as TextCNN, BERT-base, and DistillMal all achieve better classification results. In addition, the Transformer-based BERT model achieved a higher performance on the malware multi-classification task compared to the Bi-LSTM-based method, showing the advantage of Transformer in handling sequence-structured data.

To verify the effect of knowledge distillation on model performance, we conducted comparative experiments on inference performance, focusing on the number of model parameters, the model size, and the average prediction time. The experimental results are shown in Table 4.

Table 4. Comparison of the inference performance of the models.

Metrics	Parameters	Size	Avg Time
MLP	133,384	0.127 MB	0.576
TextCNN	1,970,888	1.880 MB	0.114
BERT-base	109,880,072	110 MB	9.013
DistillBert	66,760,712	63 MB	4.598
DistillMal (Our's)	1,970,888	1.880 MB	0.189

DistillMal outperformed BERT-base and DistillBert in terms of the number of parameters, the model size, and the average inference time. Compared to BERT-base, DistillMal has a significantly reduced number of parameters and a smaller model size while maintaining a high accuracy rate. Compared to DistillBert, DistillMal has a significant advantage in terms of model complexity and inference time, although there is a slight gap in accuracy. In summary, our proposed DistillMal model significantly reduces model complexity and inference time while maintaining a high classification performance, providing significant advantages for resource utilization and system efficiency in real-world deployments.

4.4. The Ablation Study

In order to obtain a good model performance, a large number of data samples are usually required. The experimental setup used to explore the impact of supervised contrastive learning tasks on model performance in cases with few data samples is as follows: The number of samples was adjusted using different levels of sample sampling treatment. For samples larger than N , only N was kept for random sampling; for samples smaller than N , the original number was unchanged. The N values were chosen as {100, 200, 300}.

From Figure 5, it can be seen that with a large sample size ($N \geq 300$), the contrastive learning module has a limited effect on model performance improvements. Notably, there was almost no improvement in accuracy on the VirusShare dataset, and the improvement in accuracy on the VirusSample dataset was relatively weak. However, when the number of training samples is insufficient, the addition of a contrastive learning task significantly enhances the model's accuracy. Notably, on the VirusSample dataset, when N was {100, 200, or 300}, the accuracy of the model improved by 2.9%, 1.4%, and 2.3%, respectively. This indicates that the effect of contrastive learning tasks on model performance is more pronounced in cases of small-sample datasets.

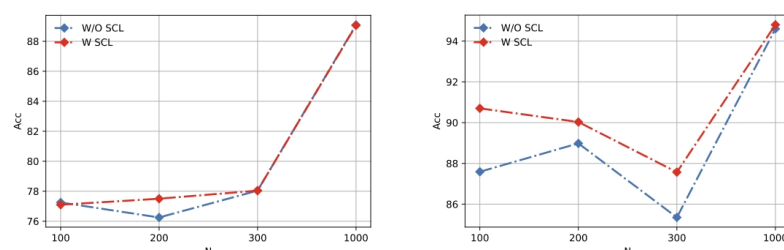


Figure 5. Effect of SCL on accuracy for different sample sizes, with the results obtained for the VirusShare dataset shown on the left and those obtained for the VirusSample dataset shown on the right.

Overall, these analytical results highlight the importance of contrastive learning tasks for small-sample datasets, especially in cases where the data are insufficient to effectively enhance the model's classification performance. At the same time, for large-sample datasets, it still helps the model to learn about class relationships, thereby improving the model's generalization ability and classification accuracy.

4.5. Parameter Sensitivity

The model has the following key hyperparameter settings: 1. In the contrastive learning task, the ratio λ of contrastive loss to cross-entropy loss is used, which determines the weight of different losses during the contrastive learning process. 2. The temperature parameter t used for label-softening in the knowledge distillation process, which smoothes the distribution of the teacher model's output and helps the student model to learn from the teacher model's knowledge. 3. During the knowledge distillation process, the ratio σ of KL loss to cross-entropy loss adjusts the value of σ , which affects the learning effect of the student model on the knowledge of the teacher model. To control variables, when conducting parameter sensitivity analysis experiments, other parameters are first set to default values, and then the parameters that are to be experimented with are modified accordingly.

In order to evaluate the impact of different λ values on the model performance, we systematically set $\lambda = 0.1, 0.3, 0.5, 0.7, 0.9$. The experimental results are presented in Figure 6.

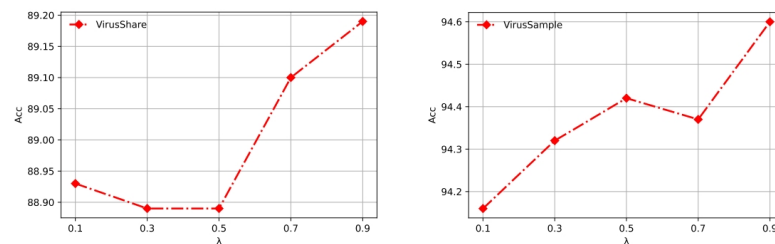


Figure 6. Effect of different λ on accuracy: the results for the VirusShare dataset are shown on the left and those for the VirusSample dataset are shown on the right.

From the results in Figure 6, it is evident that adjusting different parameters has a significant impact on the model's accuracy. Importantly, the classification performance on both datasets reached the optimal level at $\lambda = 0.9$. This outcome clearly demonstrates the effectiveness of incorporating the contrastive learning task into the fine-tuning process. Notably, a higher proportion of supervised contrastive learning loss can significantly enhance the model's detection performance.

In order to explore how the accuracy of the model varies with the distillation temperature t , we set $t = 5, 10, 15, 20$ in the parameter sensitivity analysis experiment. The experimental results are shown in Figure 7.

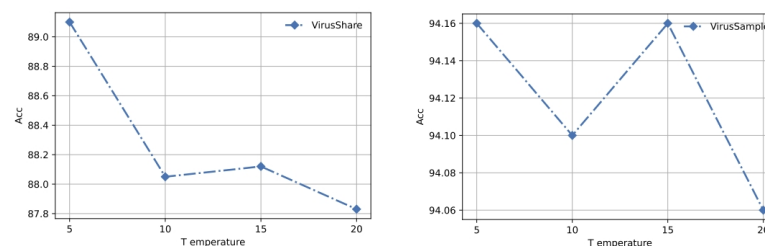


Figure 7. Effect of different λ on accuracy: the results for the VirusShare dataset are shown on the left and those for the VirusSample dataset are shown on the right.

From the graph, it is evident that as the distillation temperature increases, the model's accuracy exhibits a curve distribution resembling an "M" shape. The model achieves the highest accuracy when the distillation temperature is 5. As the distillation temperature increases to 15, there is a slight improvement in the model's accuracy. However, with further temperature increases, the model's accuracy rapidly decreases. This trend may stem from the fact that excessively high temperatures cause the student model to learn overly smooth knowledge, making it difficult to capture complex patterns and small differences in the data. This situation affects the stability and accuracy of the detection results. Therefore, in real-world applications, the judicious selection of distillation temperature is crucial.

In the experiments, the values of σ were set to $\{0.1, 0.3, 0.5, 0.7, \text{ and } 0.9\}$. The experimental results are depicted in Figure 8.

According to the analysis of the results in Figure 8, when $\sigma = 0.3$, the model achieves the best accuracy on both datasets. This indicates that the selection of an appropriate proportion of KL divergence loss and cross-entropy loss can significantly impact the model's detection performance. At a specific ratio, the student model learns both the knowledge information from the teacher model and focuses on the target task to capture the characteristics of the data, thus improving detection performance. For the VirusShare dataset, the influence of σ on model performance appears more pronounced, possibly due to the use of unbalanced datasets or large domain differences. Conversely, on the VirusSample dataset, where the dataset may be simpler or exhibit higher similarity, this influence is relatively minor.

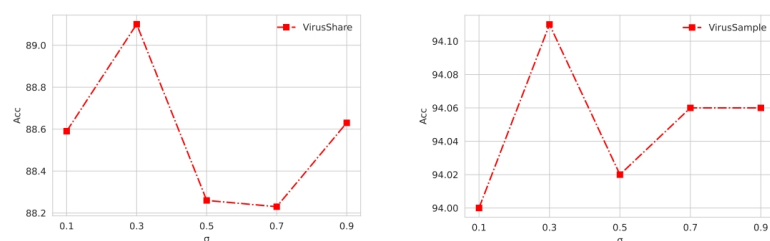


Figure 8. The effect of different σ on accuracy: the results for the VirusShare dataset are shown on the left and those for the VirusSample dataset are shown on the right.

5. Conclusions

In this paper, we proposed a lightweight malware API sequence detection model based on knowledge distillation. Our approach succeeded in building a compact and efficient student model by utilizing a large-scale pre-trained teacher model. Utilizing the knowledge distillation process allows the student network to absorb and integrate the rich knowledge of the teacher network, thus demonstrating a superior performance in malware detection tasks. We conducted experiments on two real-world datasets, VirusSample and VirusShare, and the evaluation results were encouraging, fully demonstrating the effectiveness of our approach in practical applications. Overall, our research utilized knowledge distillation to develop a lightweight model that reduces the computational and time costs while maintaining a high detection accuracy, helping to address the issues in resource-constrained device deployment malware detection. In the future, we will explore more advanced methods to deal with data imbalance situations to improve the ability of malware detection models to recognize samples from a small number of categories.

Author Contributions: Methodology, C.M.; Software, J.Z.; Formal analysis, L.K.; Resources, G.D. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the Key Technology Research and Development Program of the Zhejiang Province under Grant 2022C01125, and in part by Zhejiang Province High-Level Talent Special Support Program-Leading Talent of Technological Innovation under No. 2022R52043.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding authors.

Conflicts of Interest: The authors declare no conflicts of interest.

References

- Gopinath, M.; Sethuraman, S.C. A comprehensive survey on deep learning based malware detection techniques. *Comput. Sci. Rev.* **2023**, *47*, 100529.
- Raff, E.; Barker, J.; Sylvester, J.; Br, R.; Catanzaro, B.; Nicholas, C.K. Malware detection by eating a whole exe. In Proceedings of the Workshops at the Thirty-Second AAAI Conference on Artificial Intelligence, New Orleans, LA, USA, 2–7 February 2018.
- Arp, D.; Spreitzenbarth, M.; Hubner, M.; Gascon, H.; Rieck, K. Drebin: Effective and explainable detection of android malware in your pocket. In Proceedings of the NDSS, San Diego, CA, USA, 23–26 February 2014; Volume 14, pp. 23–26.
- Gaber, M.G.; Ahmed, M.; Janicke, H. Malware detection with artificial intelligence: A systematic literature review. *Acm Comput. Surv.* **2024**, *56*, 1–33. [\[CrossRef\]](#)
- Almakyeel, N. Deep learning-based improved transformer model on android malware detection and classification in internet of vehicles. *Sci. Rep.* **2024**, *14*, 25175. [\[CrossRef\]](#)
- Devlin, J.; Chang, M.W.; Lee, K.; Toutanova, K. Pre-training of deep bidirectional transformers for language understanding. *arXiv* **2018**, arXiv:1810.04805.
- Radford, A. *Improving Language Understanding by Generative Pre-Training*; OpenAI: San Francisco, CA, USA, 2018.
- Prima, B.; Bouhorma, M. Using transfer learning for malware classification. *Int. Arch. Photogramm. Remote Sens. Spat. Inf. Sci.* **2020**, *44*, 343–349. [\[CrossRef\]](#)
- Vasan, D.; Alazab, M.; Wassan, S.; Safaei, B.; Zheng, Q. Image-Based malware classification using ensemble of CNN architectures (IMCEC). *Comput. Secur.* **2020**, *92*, 101748. [\[CrossRef\]](#)
- Zhao, Y.; Cui, W.; Geng, S.; Bo, B.; Feng, Y.; Zhang, W. A malware detection method of code texture visualization based on an improved faster RCNN combining transfer learning. *IEEE Access* **2020**, *8*, 166630–166641. [\[CrossRef\]](#)
- Hinton, G. Distilling the Knowledge in a Neural Network. *arXiv* **2015**, arXiv:1503.02531.

12. Huang, T.; Zhang, Y.; Zheng, M.; You, S.; Wang, F.; Qian, C.; Xu, C. Knowledge diffusion for distillation. *Adv. Neural Inf. Process. Syst.* **2024**, *36*.
13. Xie, L.; Cen, X.; Lu, H.; Yin, G.; Yin, M. A hierarchical feature-logit-based knowledge distillation scheme for internal defect detection of magnetic tiles. *Adv. Eng. Inform.* **2024**, *61*, 102526. [\[CrossRef\]](#)
14. Xia, M.; Xu, Z.; Zhu, H. A Novel Knowledge Distillation Framework with Intermediate Loss for Android Malware Detection. In Proceedings of the 2022 IEEE Asia-Pacific Conference on Computer Science and Data Engineering (CSDE), Gold Coast, Australia, 18–20 December 2022; pp. 1–6.
15. Wang, Y.; Geng, Z.; Jiang, F.; Li, C.; Wang, Y.; Yang, J.; Lin, Z. Residual relaxation for multi-view representation learning. *Adv. Neural Inf. Process. Syst.* **2021**, *34*, 12104–12115.
16. Zhang, J.; Lin, T.; Xu, Y.; Chen, K.; Zhang, R. Relational contrastive learning for scene text recognition. In Proceedings of the 31st ACM International Conference on Multimedia, Ottawa, ON, Canada, 29 October–3 November 2023; pp. 5764–5775.
17. Wang, F.; Wang, Y.; Li, D.; Gu, H.; Lu, T.; Zhang, P.; Gu, N. Cl4ctr: A contrastive learning framework for ctr prediction. In Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining, Singapore, 27 February–3 March 2023; pp. 805–813.
18. Li, B.; Roundy, K.; Gates, C.; Vorobeychik, Y. Large-scale identification of malicious singleton files. In Proceedings of the Seventh ACM on Conference on Data and Application Security and Privacy, Scottsdale, AZ, USA, 22–24 March 2017; pp. 227–238.
19. Ahmadi, M.; Ulyanov, D.; Semenov, S.; Trofimov, M.; Giacinto, G. Novel feature extraction, selection and fusion for effective malware family classification. In Proceedings of the Sixth ACM Conference on Data and Application Security and Privacy, New Orleans, LA, USA, 9–11 March 2016; pp. 183–194.
20. Raman, K. Selecting features to classify malware. *InfoSec Southwest* **2012**, *2012*, 1–5.
21. Kumar, A.; Kuppusamy, K.S.; Aghila, G. A learning model to detect maliciousness of portable executable using integrated feature set. *J. King Saud Univ.-Comput. Inf. Sci.* **2019**, *31*, 252–265. [\[CrossRef\]](#)
22. Moser, A.; Kruegel, C.; Kirda, E. Limits of static analysis for malware detection. In Proceedings of the Twenty-Third Annual Computer Security Applications Conference (ACSAC 2007), Miami Beach, FL, USA, 10–14 December 2007; pp. 421–430.
23. Singh, J.; Singh, J. Detection of malicious software by analyzing the behavioral artifacts using machine learning algorithms. *Inf. Softw. Technol.* **2020**, *121*, 106273. [\[CrossRef\]](#)
24. Vinayakumar, R.; Alazab, M.; Soman, K.P.; Poornachandran, P.; Al-Nemrat, A.; Venkatraman, S. Deep learning approach for intelligent intrusion detection system. *IEEE Access* **2019**, *7*, 41525–41550. [\[CrossRef\]](#)
25. Ki, Y.; Kim, E.; Kim, H.K. A novel approach to detect malware based on API call sequence analysis. *Int. J. Distrib. Sens. Netw.* **2015**, *11*, 659101. [\[CrossRef\]](#)
26. Huang, X.; Ma, L.; Yang, W.; Zhong, Y. A method for windows malware detection based on deep learning. *J. Signal Process. Syst.* **2021**, *93*, 265–273. [\[CrossRef\]](#)
27. Catak, F.O.; Yazı, A.F.; Elezaj, O.; Ahmed, J. Deep learning based Sequential model for malware analysis using Windows exe API Calls. *PeerJ Comput. Sci.* **2020**, *6*, E285. [\[CrossRef\]](#)
28. Li, C.; Zheng, J. API call-based malware classification using recurrent neural networks. *J. Cyber Secur. Mobil.* **2021**, *10*, 617–640. [\[CrossRef\]](#)
29. Nassar F, Hubballi N. Malware detection and classification using transformer-based learning. Ph.D. Thesis, Discipline of Computer Science and Engineering, IIT Indore, Indore, India, 2021.
30. Xu, Z.; Fang, X.; Yang, G. Mal: A novel pre-training method for malware detection. *Comput. Secur.* **2021**, *111*, 102458. [\[CrossRef\]](#)
31. Paul, S.; Saha, S. CyberBERT: BERT for cyberbullying identification. *Multimed. Syst.* **2022**, *28*, 1897–1904. [\[CrossRef\]](#)
32. Jones, R.; Omar, M. Detecting IoT Malware with Knowledge Distillation Technique. In Proceedings of the 2023 Congress in Computer Science, Computer Engineering, & Applied Computing (CSCE), Las Vegas, NV, USA, 24–27 July 2023; pp. 131–135.
33. Gao, T.; Yao, X.; Chen, D. Simcse: Simple contrastive learning of sentence embeddings. *arXiv* **2021**, arXiv:2104.08821.
34. Gunel, B.; Du, J.; Conneau, A.; Stoyanov, V. Supervised contrastive learning for pre-trained language model fine-tuning. *arXiv* **2020**, arXiv:2011.01403.
35. Khosla, P.; Teterwak, P.; Wang, C.; Sarna, A.; Tian, Y.; Isola, P.; Maschinot, A.; Liu, C.; Krishnan, D. Supervised contrastive learning. *Adv. Neural Inf. Process. Syst.* **2020**, *33*, 18661–18673.
36. Düzgün, B.; Cayır, A.; Demirkıran, F.; Kayha, C.N.; Gençaydın, B.; Dağ, H. New datasets for dynamic malware classification. *arXiv* **2021**, arXiv:2111.15205.
37. Pal, S.K.; Mitra, S. *Multilayer Perceptron, Fuzzy Sets, Classification*; Indian Statistical Institute: Baranagar, India, 1992.
38. Kim, Y. Convolutional neural networks for sentence classification. *arXiv* **2014**, arXiv:1408.5882.
39. Sanh, V.; Debut, L.; Chaumond, J.; Wolf, T. Distil, a distilled version of: Smaller, faster, cheaper and lighter. *arXiv* **2019**, arXiv:1910.01108.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.